

1. Регуларни изрази

1.1. Језици

1.2. Регуларни изрази

2. Аутомати

2.1. Конатни аутомати

2.1.1. Конструкција до подкључивања

2.1.2. Без основања ентон ~~пре~~ изрази

2.2. Конструкција аутомата до регуларном изразу

2.2.1. Шомисонова конструкција

2.2.2. Тинухова конструкција

2.3. Минимизација конатних аутомата

2.3.1. Муров алгоритам

2.4. Назад у регуларни језик

2.4.1. Метод елиминације симбола

2.5. Пресек два аутомата

3. Антиаксна анализа

3.1. Конвертовање у LL(1) - ~~Наше~~

3.1.1. Уклањање терозитивних симбола

3.1.2. Уклањање редосишних симбола

3.1.3. Уклањање E-прелима

3.1.4. Уклањање једносржних прелима

3.1.5. Уклањање неке рекурзије

3.1.6. Уклањање посредне неке рекурзије

3.1.7. Уклањање неке факторизације

3.2. LL(1) Граматика таблице

3.2.1. Одредивање First(x) скјула

3.2.2. Одредивање Follow(A)

3.2.3. Одредивање скјула избора

3.2.4. Таблице изенаска

3.3. SLR(1) Граматика таблице - ~~Наше~~

3.3.1. Конатни аутомат SLR(1)

3.3.2. Таблице изенаска SLR(1) или ACTION-GOTO

3.3.3. Пример изрази a + * , ()

4. Семантичка анализа

4.1. Атрибутивне граматике

4.2. Силаба алгебраичке синтаксе

Аутоматии

- Формални модели израчунавања који омогућавају претпостављање језика, исцрпљивање да ли дата рет припада неком језику.

* Класе аутоматиа:

1. Конечни аутоматии - за регуларне језике.

- Регуларни језик је најмањи скуп језика над Σ , важи:
 - празан језик је регуларан
 - језик $L_\epsilon = \{\epsilon\}$ је регуларан.
 - за сваки симбол a из Σ , језик $\{a\}$ је регуларан.
 - ако су L_1 и L_2 регуларни, регуларан је и онај добијен њиховим дописивањем и унујом.
 - ако је L регуларан онда је и Книтујево затварање тог језика регуларан.

Скратено: Регуларни језици су само они који се могу добити понављањем из језика $\emptyset$, L_ϵ и $\{a\}$ и применом ограничене броја операција унује, надовезивања и Книтујевог затварања.

2. Потпуни аутоматии - за контекстно слободне језике.

- Контекстно слободни језици - добијени из KSG (контекстно слободна граматика) која је уређена мрежа $G = (\Sigma, N, S, P)$:
 - Σ - азбука над којом се ради граматика (терминали)
 - N - коначни скуп незадермних симбола (нестерминали)
 - S - почетни нестерминали ($S \in N$)
 - P - коначни скуп правила извођења ($A \rightarrow a, A \rightarrow aSb$ извођа)
- нпр. $\Sigma = \{a, b\}$, $N = \{S\}$ и $P = \{S \rightarrow aSb, S \rightarrow \epsilon\}$, или:
- $$\begin{array}{l} S \rightarrow aSb \\ \quad | \epsilon \\ \quad ; \end{array}$$

Ови језици не морају бити регуларни. Препознати пример је језик $L(G) = \{a^n b^n \mid n \geq 0\}$, што није регуларан језик. Сваки регуларан језик је контекстно слободан.

3. Линеарно ограничени аутоматии - за контекстно зависне језике (пример $aSb \rightarrow aSb$, S постаје a ако a је окружено са a и b)

4. Потпуни аутоматии са два симбола - еквивалентни Тјуринговим машинама.

Конечни аутомати

- Могу се представити у облику графа, чворови су стања а усмерене грање су прелазни који су означени симболичком азбуком.

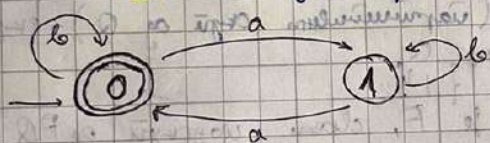
Почетно стање је чвор представљен као чвор у који улази грања која не креће ни из једног другог стања.

Завршно стање је чвор представљен са два кружића.

- Примером провере да ли реч припада језику вратимо се кроз реч и посматрамо коју грању можемо да праћемо да би прешли у следеће стање, или обрнуто радимо из следећег стања док не дођемо до краја речи, ако се налазимо на завршном стању онда реч припада језику.

- Конечни аутомат је представљен уређеном петорком $A = (\Sigma, Q, I, F, \Delta)$.

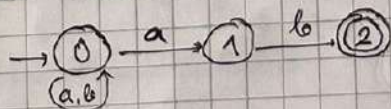
- Σ - азбука аутомата
- Q - конечни скупи стања
- I - почетна стања
- F - завршна стања
- Δ - скупи прелазна



$w = ababab$; $a? (0,1)$, $b? (1,1)$, $a? (1,0)$, $b? (0,0)$, $b? (0,0)$

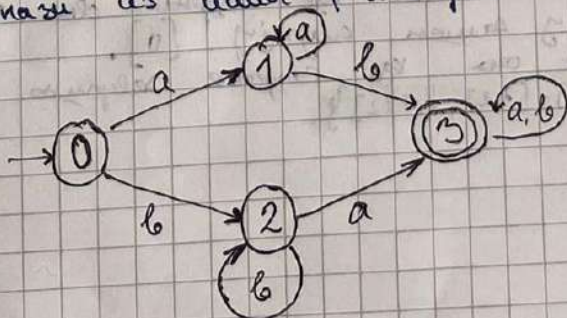
⇒ на крају се налазимо на стању 0 које је и завршно, дакле реч припада језику аутомата.

- Недетерминистички конечни аутомат - уколико имамо две грање које можемо да истражимо да дођемо у следеће стање.



признаје две речи које се завршавају на ab .

- Полијутни детерминистички конечни аутомат - уколико за сваки терминал има поседна грања која може да се истражи до следећег стања, свако стање има једну грању која излази из њега (и нарочито није недетерминистички).



Овај аутомат представља две речи које садрже бар једном словом a и бар једном словом b .

Ситње реше или незаверљиво ситње - у овом ситњу се унапред изгледа одозлогавно карактеристика из некое ситње. Сваки неопредељен аутомат може се представити у попуњеном ситњу.

Конечни аутомат је детерминистички ако су испуњени следећи услови:

- свакој ситњи једно постојеће ситње
 - нема ϵ -прекоза
 - за $q \in Q$ и $a \in \Sigma$ постоји највише једно ситње $q' \in Q$ такво да је $(q, a, q') \in \Delta$.
- = из сваког ситња постоји једно ситње. Наиме да се иде у следеће из некоег терминала, а попуњено је ако постоји из сваког ситња за сваки терминал.

Конструкција из попуњеног

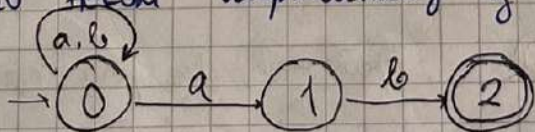
Из недетерминистичког A аутомата грађемо попуњен детерминистички $A' = (\Sigma, PQ, \{1\}, F, \delta)$:

- скупи ситња A' је PQ (партиципални скупи од Q) - скупи свих попуњеног од Q .
- скупи постојећих ситња $\{1\}$.
- скупи заверљивих ситња је F , сваки подскуп од PQ који садржи бар једно заверљиво ситње аутомата A .
- функција пренаома аутомата A' је дефинисана као:

$$\delta(R, a) = \{q \mid \text{свако } r \in R, r \xrightarrow{a} q\}$$

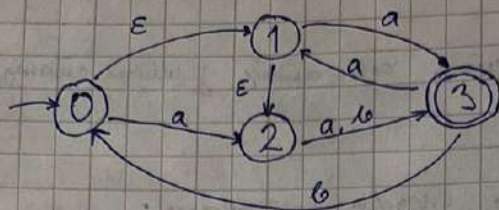
Аутомат

- Термином два ситња, партиципалан скуп од Q , где се налазе и постојеће и сва заверљива.
- Прекози се одређују тако што узмемо два ситња и идемо по некоем терминалу у која се ситња можемо да одемо.



Идемо два ситња. $Q = \{0, 1, 2\} \rightarrow PQ = \{[1], [0], [1], [2], \dots\}$, где је 2^{101} .

Почетно ситње је ситње које садржи два постојећа ситња попуњеног аутомата, у овом случају $[0]$.
 Сва заверљива ситња су оне које садрже заверљиво ситње попуњеног аутомата $F = \{[2], [02], [12], [012]\}$.



Пралічым ϵ -заключэнне:

$E(0) = \{0, 1, 2\}$ - іх усе станы

$E(1) = \{1, 2\}$

$E(2) = \{2\}$

$E(3) = \{3\}$, або су нова станна $E(0), E(1), E(2), E(3)$

Правілы:

$(E(0), a) = \{E(2), E(3)\}$, ад 0 ідзе па 'a' у 2, ад 1 у 3 і ад 2 у 3

$(E(0), b) = \{E(3)\}$

$(E(1), a) = \{E(3)\}$

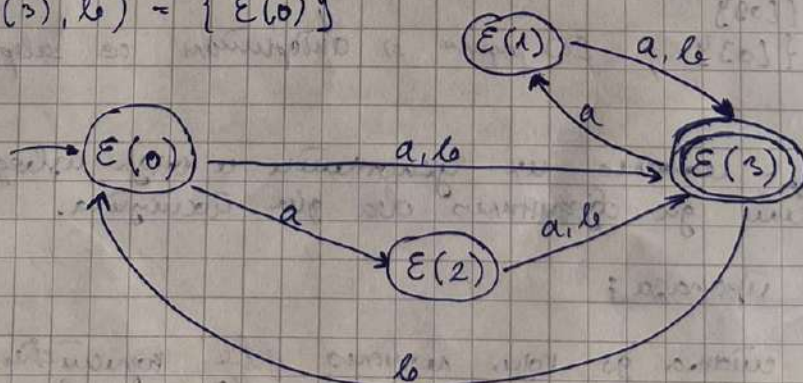
$(E(1), b) = -||-$

$(E(2), a) = -||-$

$(E(2), b) = -||-$

$(E(3), a) = \{E(1)\}$

$(E(3), b) = \{E(0)\}$



Забіцены аўтамат ёсць недэterminістычны іа мораць іх утварыць аўтамат.

Конструкција аутомата по регуларном изразу

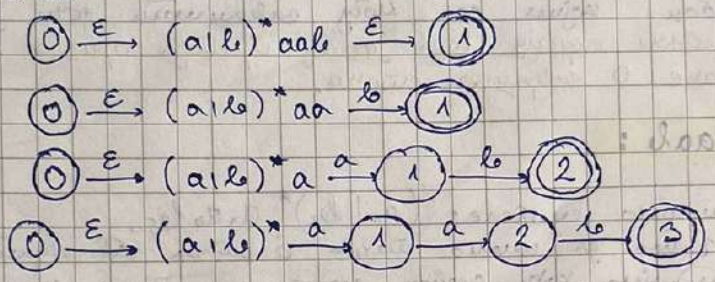
Сваки аутомат описан регуларним изразом може представљати неки коначни аутомат.

Помислова конструкција

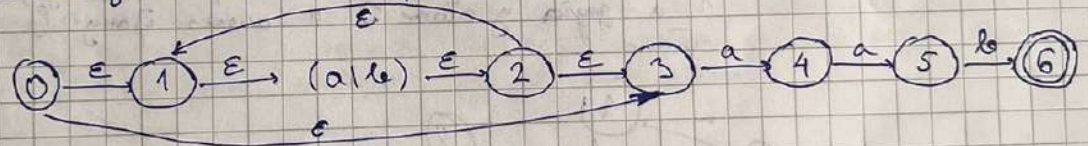
- Аутомат је нормализован ако:
 - Има тачно једно завршно стање
 - Ни један лук му не води у почетно стање
 - Из сваког стања излази или тачно један лук са етикетом из алфабета Σ или највише два лука са етикетом ϵ .

• Помислова конструкција за ретек $(a|b)^*aab$:

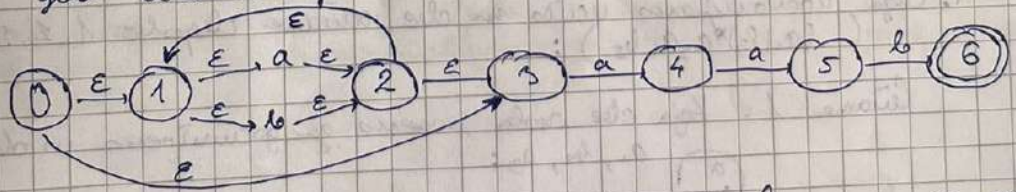
Прво се решавамо „слободних“ слова или „петља“, у нашем случају aab :



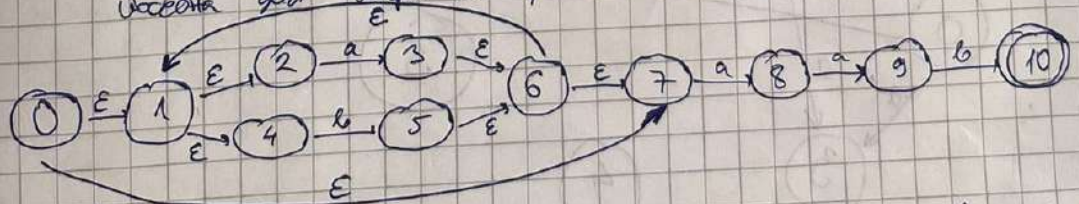
Водимо рачун да одржимо нормализован аутомат. Сада издвајамо $(a|b)^*$, + унели смо петљу а за * унели смо петљу и око на крај петље:



Приметимо да морамо да додамо додатно стање да не постоји слобода да би садржали нормализован аутомат (због друге тачке дефиниције нормализованог аутомата) као и што смо морали да додамо посебно стање након (a|b) (због прве тачке). Дакле рачунајуемо а|b издвајањем две етикетне гране из слова 1 (ово је из прве тачке):

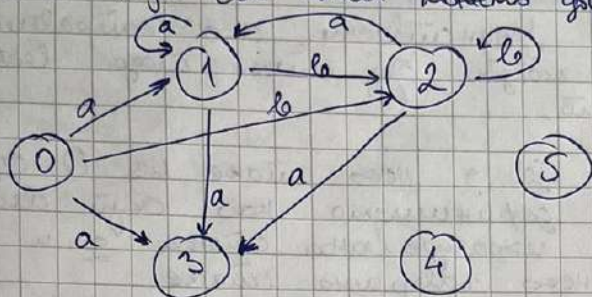


Коначно да би направили гране за а и б морали да додмо поседна два слова и уведемо између њих за свако слово:



Након што смо направили аутомат треба да изведемо аутомат конструкције по подок да би генерализовали аутомат.

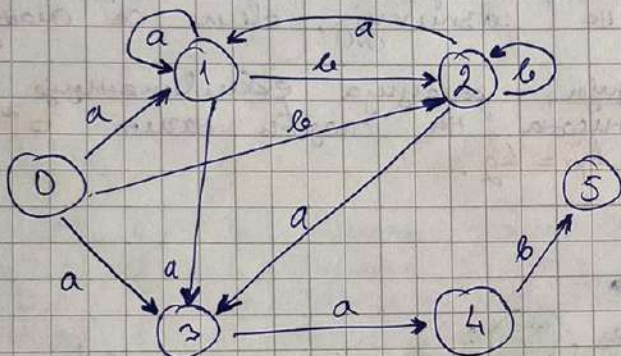
Слика 2: Која две слова можемо додати у овој слици: a_1, a_2, a_3 :



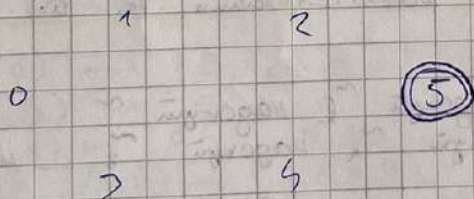
Слика 3: Директно можемо додати само до: a_4

Слика 4: Директно само у слици: b_5

Слика 5: У овој слици не можемо додати ни до којег дрвца



5. Сада нас занима која су то сликања која могу бити завршена, израчунамо који словом може да се завршава реч језика $(a_1/b_2)^* a_3 a_4 b_5$. У нашем случају видимо да то мора бити слово b_5 , нема ниједног дрвца кандидата.



* По завршетку аутомата треба да применимо алгоритам конструкције аутомата. По подставама да би добили дискретизован

Минимизација коначних аутомата

- Добити РДКА или јединствени, ако направимо да неки РДКА има најмањи могући број стања иако већ добијемо јединствени аутомат.
- Језик стања - L_A , где је A неко стање из Q аутомата $A = (\Sigma, Q, I, F, \delta)$, дефинисано као скуп свих речи које су етикетисане неком изражавањем стања q и одакле можемо доћи у неко завршно стање.
- Ако у аутомату за нека два стања p и q важи $L_p = L_q$ онда знамо да се њих не разликује у аутомату почевши од стања p и q .
- Минимизација - уједињава верни стањане два или више стања која се не разликују, ови се називају стања.
- Неродова еквиваленција - релација еквиваленције међу стањима аутомата дефинисана на следећи начин $p \sim q$ што је исто што и $L_p = L_q$.

Муров аутомат

Итеративно одређивање $(\sim)$:

- Нека је $L_p^k = \{w \mid w \text{ из } L_p \text{ и } |w| \leq k\}$
- Нека је $p \sim_k q$ исто што и $L_p^k = L_q^k$

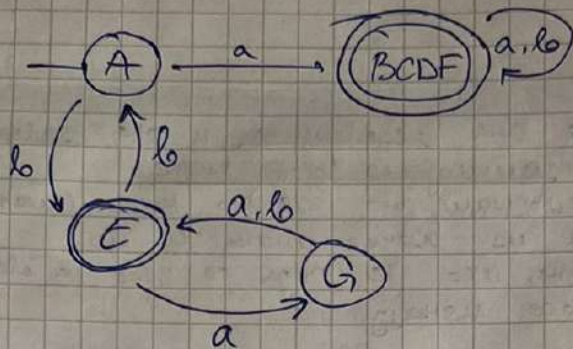
Важни својства:

- За свако k из $\mathbb{N}$, $p \sim q \Rightarrow p \sim_k q \Rightarrow p \sim_{k+1} q$ (ако се нека два језика не разликују то је једнај рачуна онда се неће разликовати то и једнај рачуна дужине k).

$\forall k \in \mathbb{N}$, $\sim_k$ надскрућу $\sim_{k+1}$ надскрућу $\sim_{k+2}$ -- надскрућу $\sim$

- $\sim$ је пресек свих релација $\sim_k$ где $k \geq 0$.
 $p \sim q$ стања p и q се не разликују до рачуна дужине 0.
 $p \sim_1 q$ исто што и $p \sim_0 q$ и за та из азбуке важи $\delta(p, a) \sim_0 \delta(q, a)$.
 $p \sim_2 q$ исто што и $p \sim_1 q$ и та из азбуке важи $\delta(p, a) \sim_1 \delta(q, a)$.
- $p \sim_k q \Rightarrow p \sim_{k+1} q$ и за та из азбуке важи $\delta(p, a) \sim_k \delta(q, a)$

Полицик изабривањем док две класе не престану да се не разликују.



Обичајно аутомати називамо Минимални потпуни детерминистички коначни аутомат (МРДКА).

Циљ аутомата је уклањање некорисних стања да би се аутомат што лакше анализирао.

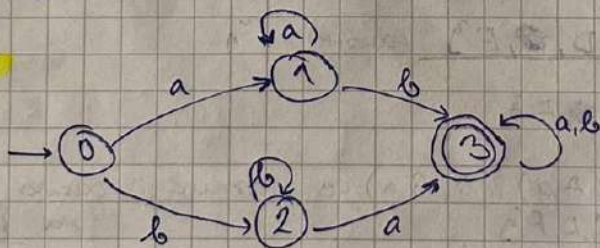
Назад у регуларни језик

Сваки регуларни језик је представљив, важи и обрнуто за сваки коначни аутомат постоји регуларни израз који га описује (Нису јединствени).

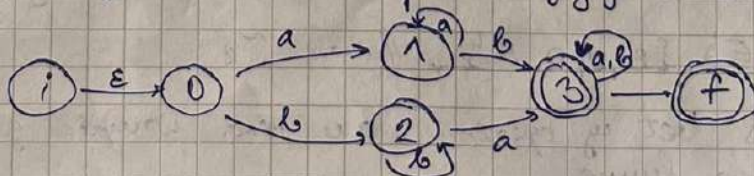
Метод елиминације стања

* Додајемо једно ново почетно стање "i", повезујемо га са почетним стањем аутомата елиментом празног, и једно ново коначно стање "f", повезујемо са елиментом празног на коначно стање аутомата. Уклањамо једно по једно стање све док не остану само стања i и f.

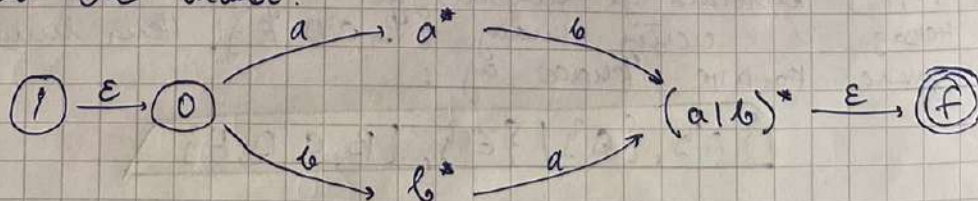
Пример



Прво као што смо већ рекли додајемо посебна нова стања i и f:



Уклањамо све остале:

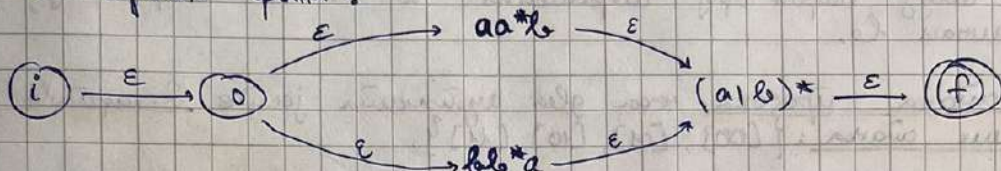


* Слика 1: има путеву а што значи да се а може у свакој ситуацији поновити 0 или више пута дакле пишемо a^* .

* Слика 2: слично као слика 1, имамо b у путеву па пишемо b^* .

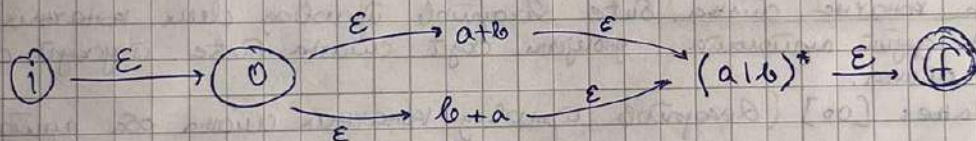
* Слика 3: посматрајмо прво само путеву a и b , дакле етикета путеву може да се напише и као $(a|b)$, дакле јер је путева што значи да може да се понови 0 или више пута па пишемо $(a|b)^*$.

Решимо се горњих путева:

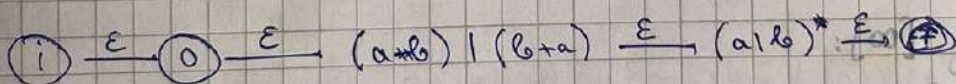


Како из слика 0 и 1 имамо путеву са етикетом "а" значи да је само a^* може имати једно "а", дакле aa^* , у претходно из a^* у $(a|b)^*$ имамо путеву са етикетом b па можемо да закључимо да неки a^* мора постојати бар једном b . Добивамо aa^*b ово можемо написати и као $a+ b$ јер значи да се "а" мора бар једном појавити на путеву.

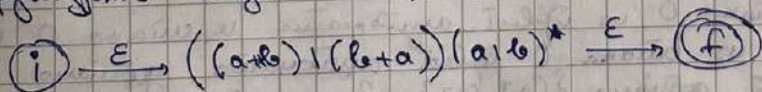
Иако се и дешава и на горњим путевима:



Стајемо горњу и доњу путеву:



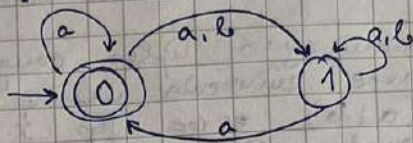
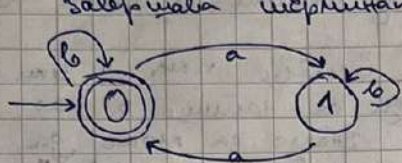
Придружујемо сада два дела слика:



• Коначно добијемо ретекс $((a+b)|(b+a))(a|b)^*$.

Пресек два аутомата

- Ако смо да направимо пресек аутомата који представља рети које морају да имају паран број терминала "а", и аутомата који мора да се завршава терминалом "а":

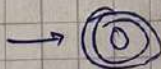


- Пресек два два аутомата даје аутомат који представља оне рети које имају паран број терминала "а" и не завршавају се на терминал "б".

- Ова сета пресека је декартов производ нихових сета: $\{[00], [01], [10], [11]\}$.

- Када будемо посматрали преносе из сета $[xy]$ ћемо декартов производ сета која се добију из сета x које припада првом аутомату и из сета y које припада другом аутомату.
- За свако ново сета морамо да одредимо и нихов преносе.
- Ова могућа почетна сета буде декартов производ двех почетних сета првог и другог аутомата, добити оку сета буде погледу двех могућих сета.
- Могуће почетне сете: $[00]$ (декартов производ почетних сета оба аутомата).

- Ова могућа конечна сета буде декартов производ двех конечних сета првог и другог аутомата, добити оку сета буде погледу двех могућих сета.
- Конечна сете: $[00]$ (декартов производ конечних сета оба аутомата).

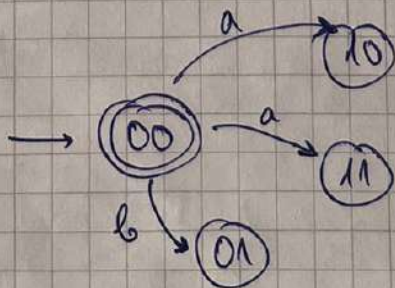


→ Почнимо од $[00]$:

$$\delta([00], a) = \{ \delta([0], a) \times \delta([0], a) \} = \{ \{1\} \times \{0, 1\} \} = \{ [10], [11] \}$$

Сета 00 добија се из сета 0 из првог аутомата и сета 0 из другог аутомата, како из сета 0 првог аутомата добијемо скупу $\{[1]\}$, док из другог добијемо $\{[0], [1]\}$ декартов производ ова два сета даје нам $\{[10], [11]\}$.

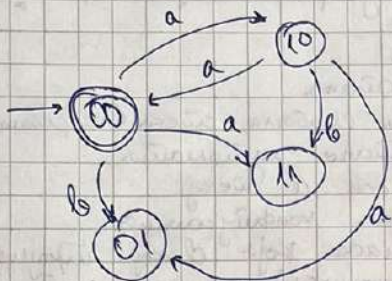
$$\delta([00], b) = \{ \delta([0], b) \times \delta([0], b) \} = \{ \{0\} \times \{0\} \} = \{ [00] \}$$



→ Посчитано **[10]**:

$$\delta([10], a) = \{ \delta([1], a) \times \delta([0], a) \} = \{ \{0\} \times \{0\} \} = \{[00], [01]\}$$

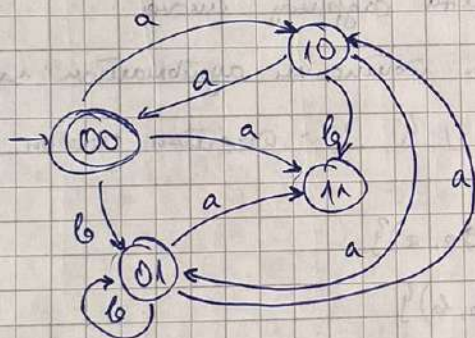
$$\delta([10], b) = \{ \delta([1], b) \times \delta([0], b) \} = \{ \{1\} \times \{1\} \} = \{[11]\}$$



→ Посчитано **[01]**:

$$\delta([01], a) = \{ \delta([0], a) \times \delta([1], a) \} = \{ \{1\} \times \{0, 1\} \} = \{[10], [11]\}$$

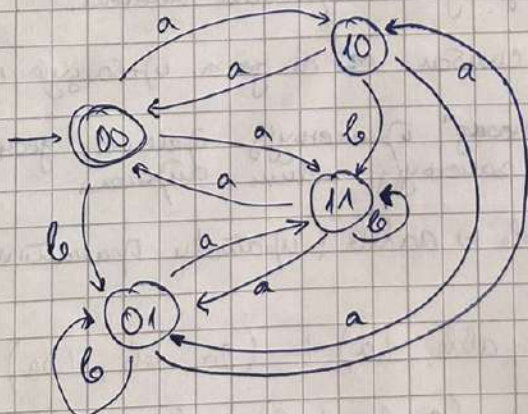
$$\delta([01], b) = \{ \delta([0], b) \times \delta([1], b) \} = \{ \{0\} \times \{1\} \} = \{[01]\}$$



→ Посчитано **[11]**:

$$\delta([11], a) = \{ \delta([1], a) \times \delta([1], a) \} = \{ \{0\} \times \{0, 1\} \} = \{[00], [01]\}$$

$$\delta([11], b) = \{ \delta([1], b) \times \delta([1], b) \} = \{ \{1\} \times \{1\} \} = \{[11]\}$$



Обах автоматах бы все равно **детерминизовано и минимизовано.**

Синтаксна анализа

- * Ова синтаксна анализа (парсирање) јесте да за дају реч $w \in \Sigma^*$ и дају граматичку G конструише извођење реч w у граматичку G по табели.

Потисни аутомат - уређена седморка $A = (\Sigma, Q, \Gamma, q_0, Z_0, K, \delta)$:

- Σ - коначна азбука
- Q - коначни скуи стања
- Γ - коначна азбука симбола тиска (терминали и не терминали)
- q_0 - почетно стање аутомата
- Z_0 - почетни симбол а тиска
- K - скуи завршних конфигурација
- δ - функција преласка која даје прелаз из $Q \times \Sigma \times \Gamma^*$ одговарајући коначни скуи парова из $Q \times \Gamma^*$.

Конфигурација потисног аутомата одразила уређену тројку (q, w, ρ) :

- q - тренутно стање
- w - реч која следи на улазу
- ρ - тренутни садржај тиска

Како би изнекао ред a потисним аутоматом за граматичку $S \rightarrow aSb \mid \epsilon$;

Овде је $\Gamma = \{a, S, b, \perp\}$, $\perp$ - почетни симбол, функција преласка би била:

$$\delta(q_0, a, \epsilon) = \{(q_0, a)\}$$

$$\delta(q_0, b, \epsilon) = \{(q_0, b)\}$$

$$\delta(q_0, \epsilon, aSb) = \{(q_0, S)\}$$

$$\delta(q_0, \epsilon, \epsilon) = \{(q_0, S)\}$$

$$\delta(q_0, \epsilon, \perp S) = \{(q_0, \epsilon)\}$$

Овакве аутомат прелазе јеzik празним тиска.

- * Прелазни преобликовања - симбол се a улаз преобликује на тиска (прва злат прелаз).

- * Прелазни редукције - "улаз" примењују правила, десне стране са леве тиска се заистују левим симбола.

Пример: $S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabbb$ (праксни граматичку), аутоматом би то изнекао:

$$(q_0, aabbb, \perp) \vdash (q_0, abbb, \perp a) \vdash (q_0, bbb, \perp aa) \vdash (q_0, bb, \perp aas)$$

$$\vdash (q_0, b, \perp aasb) \vdash (q_0, b, \perp aS) \vdash (q_0, \epsilon, \perp aSb) \vdash (q_0, \epsilon, \perp S)$$

$$\vdash (q_0, \epsilon, \epsilon)$$

Велики проблем је недетерминисан узрокован елиминацијом премака, овај проблем се решава увођењем маркера на крај улаза односно $\rightarrow$, који се догађа у азбуку и на који се детерминисално да се увек налази на крају речи и само ту.

Формално овај модификација се уводи проширивањем граматике (G') .

$$L(G') = \{ w \rightarrow \mid w \in L(G) \}$$

Пример: $S \rightarrow aSb \mid \epsilon$, проширена граматика: $S' \rightarrow S \rightarrow$, $S \rightarrow aSb \mid \epsilon$.

Недетерминистички аутомат $L(G')$:

$$\delta(q_0, \epsilon, S') = \{(q_0, S \rightarrow)\}$$

$$\delta(q_0, \epsilon, S) = \{(q_0, aSb), (q_0, \epsilon)\}$$

$$\delta(q_0, a, a) = \{(q_0, \epsilon)\}$$

$$\delta(q_0, b, b) = \{(q_0, \epsilon)\}$$

$$\delta(q_0, \rightarrow, \rightarrow) = \{(q_0, \epsilon)\}$$

Да би детерминистички овај аутомат уводимо стања q_a, q_b и $q_{\rightarrow}$.

$$\delta(q_a, a, z) = \{(q_a, z)\}, \quad \forall z \in \Gamma = \Sigma \cup N$$

$$\delta(q_b, b, z) = \{(q_b, z)\}, \quad \forall z \in \Gamma = \Sigma \cup N$$

$$\delta(q_{\rightarrow}, \rightarrow, z) = \{(q_{\rightarrow}, z)\}, \quad \forall z \in \Gamma = \Sigma \cup N$$

$$\delta(q_a, \epsilon, S') = \{(q_a, S \rightarrow)\}$$

$$\delta(q_a, \epsilon, S) = \{(q_a, aSb)\}$$

$$\delta(q_b, \epsilon, S) = \{(q_b, \epsilon)\}$$

$$\delta(q_{\rightarrow}, \epsilon, S') = \{(q_{\rightarrow}, S \rightarrow)\}$$

$$\delta(q_{\rightarrow}, \epsilon, S) = \{(q_{\rightarrow}, \epsilon)\}$$

$$\delta(q_a, \epsilon, a) = \{(q_0, \epsilon)\}$$

$$\delta(q_b, \epsilon, b) = \{(q_0, \epsilon)\}$$

$$\delta(q_{\rightarrow}, \epsilon, \rightarrow) = \{(q_0, \epsilon)\}$$

⊗ Цело парсирање и поштоматни аутомат - цело парсирање или синтаксна анализа нитице.

$$S \rightarrow E \rightarrow$$

$$E \rightarrow E + T$$

$$\mid T$$

$$T \rightarrow T * F$$

$$\mid F$$

$$F \rightarrow (E)$$

$$\mid a$$

Аутомат би изгледао:

$$\begin{aligned}\delta(q_0, \epsilon, S) &= \{(q_0, \epsilon - 1)\} \\ \delta(q_0, \epsilon, E) &= \{(q_0, \epsilon + T), (q_0, T)\} \\ \delta(q_0, \epsilon, T) &= \{(q_0, T * F), (q_0, F)\} \\ \delta(q_0, \epsilon, F) &= \{(q_0, (E)), (q_0, a)\} \\ \delta(q_0, a, a) &= \{(q_0, \epsilon)\} \\ \delta(q_0, (, () &= \{(q_0, \epsilon)\} \\ \delta(q_0, (,) &= \{(q_0, \epsilon)\} \\ \delta(q_0, +, +) &= \{(q_0, \epsilon)\} \\ \delta(q_0, *, *) &= \{(q_0, \epsilon)\} \\ \delta(q_0, -, -) &= \{(q_0, \epsilon)\}\end{aligned}$$

Конвертовање у LL(1) (1)

LL(1) Граматика не сме бити вишезначна:

1. Важно правило о приоритету знакова
2. Оператор ниже приоритета се налази на вишем нивоу
3. Оператори више приоритета се налазе ниже у Граматици
4. Оператори на истом нивоу имају ист приоритет.

Дефиницено језик који представља $+$, $-$, $*$ и заграда, различито црво које су од ових оператора најнижи нивоа: $+$ и $-$ дакле они су најниже у језику, $*$ је одмах изнад а заграда највише нивоа:

$$\begin{array}{lcl} E & \longrightarrow & E + T \\ & | & E - T \\ & | & T \\ T & \longrightarrow & T * F \\ & | & F \\ F & \longrightarrow & (E) \\ & | & a \end{array}$$

Битно је напоменути да оператори истог приоритета морају имати ист асоцијативност иначе долази до вишезначности.

LL(1) Граматика не сме да има некорисне симболе - A и X некорисни ако не постоји ни једно извођење које произведе w правилно.

Уклањање непродуктивних симбола

- Одређено скупи корисних
- Ако постоји правилно $A \rightarrow w$ где је w из Σ^* , додајемо A у скупи
- Ако постоји правилно $A \rightarrow X_1 X_2 \dots X_n$, додајемо A у скупи терминали или скуп продуктивних, додајемо A у скупи
- Када одређено скупи продуктивних симбола издајемо оде оне које не

Пример

$S \rightarrow AB$
 $\mid CA$
 $A \rightarrow a$
 $B \rightarrow ABD$
 $\mid EA$
 $C \rightarrow aB$
 $\mid b$
 $D \rightarrow aC$
 $E \rightarrow BA$

Објашњење

Формуло две грамике
која имају непродуктивне
симболе

$\Rightarrow$

$S \rightarrow CA$
 $A \rightarrow a$
 $C \rightarrow b$
 $D \rightarrow aC$

Процес још једном
извесити да ли грамике
да није случајно настао
новим непродуктивним.

Уклањање недостижних симбола

- Прво уклањамо непродуктивне.
- Формирамо скуп достижних
- S додајемо у достижне.
- Ако је A у скупу достижних и постоји грамико $A \rightarrow \alpha B \beta$ тада
и B припада скупу.
- Када одредимо достижне елиминисамо све који нису у скупу.

Пример

$S \rightarrow CA$
 $A \rightarrow a$
 $C \rightarrow b$
 $D \rightarrow aC$

$\Rightarrow S$ додајемо одмах $\{S\}$, како је S у достижним а он садржи
 C и A одмах додајемо и C и A у достижне $\{S, A, C\}$ како ни
 C ни A не показују на D , онда не треба D да показујемо:

$S \rightarrow CA$
 $A \rightarrow a$
 $C \rightarrow b$

Уклањање ϵ -грамике

- Формирамо скуп свих анулирајућих симбола
- Ако $A \rightarrow \epsilon$, A додајемо у скупу
- Ако $A \rightarrow \alpha$, а сви симболи из α припадају скупу анулирајућих
додајемо и A .

⊗ Језик је еџипон слободан ако не садржи ни једно еџипон
грамико или ако не садржи ниједно сеп у поштом
симболу.

Пример $S \rightarrow aSb$
 $\quad \quad \quad | \epsilon$

Ејаксон се наводи у почетном или се наводи и са десне стране правила па морало напредовати ново правило:

$$S' \rightarrow S$$

$$\quad \quad \quad | \epsilon$$

$$S \rightarrow aSb$$

$$\quad \quad \quad | ab$$

Пример $S \rightarrow ABC$

$$A \rightarrow CAC$$

$$\quad \quad \quad | C$$

$$C \rightarrow aC$$

$$\quad \quad \quad | \epsilon$$

$$B \rightarrow bC$$

Прво додјемо C јер он има правила које садржи ϵ $\{C\}$, како A садржи правило које директно има C додјемо и A $\{AC\}$, сада конструишемо $S \rightarrow ABC | BC | AB | B$, ово урадио и за остале:

$A \rightarrow CAC | CA | CC | AC | C$, $C \rightarrow aC | a$, $B \rightarrow bC | b$. Овде настају једносмукна правила.

• Правило је једносмукно ако $A \rightarrow B$, а и A и B су термини.

Уклањање једносмукних правила

- Најлакше размислити да буде ејаксон слободна.
- За сваки термин A одредимо скуп $JA = \{B \mid A \Rightarrow^* B\}$.
- За сваки термин A , скупи JA недалних правила заменимо скупом правила који настају пото што за свако B из JA узимо десне стране оних не-једносмукних правила симбола B из оригиналне граматике.

Пример

$$S \rightarrow ABC | BC | AB | B$$

$$A \rightarrow CAC | CA | CC | AC | C$$

$$C \rightarrow aC | a$$

$$B \rightarrow bC | b$$

Скупи $JS = \{S, B\}$, $JA = \{A, C\}$, $JC = \{C\}$ и $JB = \{B\}$. Новонастали граматика изгледа:

$S \rightarrow ABC \mid BC \mid AB \mid BC \mid \epsilon$
 $A \rightarrow CAC \mid CA \mid \epsilon \mid AC \mid aC \mid a$
 $C \rightarrow aC \mid a$
 $B \rightarrow bC \mid b$

Овде можемо применити неку рекурзију изграђивања $A \rightarrow AC$,
 јер $A \rightarrow Ad$ је нека рекурзија.

Уклањање неке рекурзије

- Посматрамо грамматика A неизграђива, и бирамо оде грамматика која не садржи рекурзију у себи.
- На оде грамматика напомена A' .
- грамматика на неизграђива A' , која не садржи оде грамматика која је рекурзивно из покушај A оде грамматика и уклањање неизграђива A и оде напомена A' , оде рекурзија грамматика оде грамматика ејасно.

$A \rightarrow Ad_1 \mid Ad_2 \mid \dots \mid Ad_n \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_m \leftarrow \epsilon$

грамматика:

$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_m A'$

$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_n A' \mid \epsilon$

Пример

$E \rightarrow E + T$
 $\mid T$

$T \rightarrow T * F$
 $\mid F$

$F \rightarrow (E)$
 $\mid a$

$\Rightarrow$ Применимо неку рекурзију E и T :

$E \rightarrow TE'$

$E' \rightarrow +TE'$
 $\mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT'$
 $\mid \epsilon$

$F \rightarrow (E)$
 $\mid a$

Уклањање посредне илве рекурзије

- Прво поређано све неутерминисано у низ $A_1, A_2, A_3, \dots, A_n$ изразимо се да поредак буде такав да X иде пре Y кад год постоји $X \rightarrow Y$.
- Формирамо прате од A_i до A_j где $i < j$ постоји пралмно $A_i \rightarrow A_j$ ако нема петљи ни повратних права (A_i до A_j где $i \geq j$), т.е. постоји цела рекурзија.
- Поларизује права и петље симултаном са целом у десно.
- Изгуби да ни имамо пралмно $A_i \rightarrow A_j$ за $i=1, 2, 3, \dots, n$, ако постоји права се A_j пралмно заместује у A_i пралмно где $A_i \rightarrow A_j$.

Пример

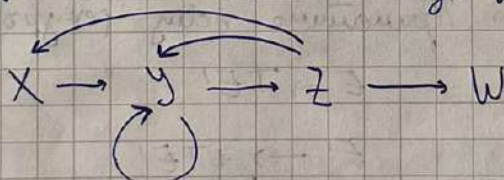
$$X \rightarrow Yb \mid aZ$$

$$Y \rightarrow Ya \mid Zb$$

$$Z \rightarrow Wa \mid Xb \mid Yc \mid c$$

$$W \rightarrow aW \mid b$$

Како X показује на Y знамо да мора бити пре X и Y .
 Пошто Y показује на Z на знамо да је пре Z дакле X, Y, Z ,
 Z показује на W на знамо да је пре W .



Приметимо да кад Y лет постоји цела рекурзија коју треба да рецимо прво:

$$Y \rightarrow Zb \mid Y'$$

$$Y' \rightarrow aZ' \mid \epsilon$$

- уклањамо права Y у Y .

Затим оснаћујемо Z :

$$Z \rightarrow Wa \mid Yb \mid aZb \mid Yc \mid c$$

$$Z \rightarrow Wa \mid ZbYb \mid aZb \mid ZbYc \mid c$$

- овим уклањамо повратну права од Z ка X .

Рецимо се цели рекурзије:

$$Z \rightarrow WaZ' \mid aZbZ' \mid cZ'$$

$$Z' \rightarrow bY'cZ' \mid bY'bZ' \mid \epsilon$$

- уклањамо права Z у Y .

W свакако није имао повратне права шаво да се олеге поларизације.

$X \rightarrow Yb \mid aZ$
 $Y \rightarrow ZbY'$
 $Y' \rightarrow aY' \mid \epsilon$
 $Z \rightarrow WaZ' \mid aZbZ' \mid aZ'$
 $Z' \rightarrow bY'cZ' \mid bY'cbZ' \mid \epsilon$
 $W \rightarrow aW \mid b$

Укљонање неких факторизација

• Ако постоји правилно облика $A \rightarrow \alpha p_1 \mid \alpha p_2 \mid \dots \mid \alpha p_k$, где је α симбол која је сасијалована из терминала и не терминала.

• Онда важи $A \rightarrow \alpha A'$
 $A' \rightarrow p_1 \mid p_2 \mid \dots \mid p_k$

⊛ Дефиницијом скупа $\text{First}(x) = \{a \mid x \Rightarrow^* aW, W \in \Sigma^*\}$ где је $x \in (\Sigma \cup N)^*$,

и скупа $\text{Follow}(A) = \{a \mid S \Rightarrow^* \alpha A a \beta \text{ где је } A \in N \text{ и } \alpha, \beta \in (\Sigma \cup N)^*\}$.

1.1) Правила уградње (2)

Грешке First(x) скупа:

• Ако је $x = a$ из Σ , тада $\text{First}(x) = \{a\}$

• Ако је $x = A$ из N , тада:

1. Дефиниран граф који је елемент из $\Sigma \cup N$.

2. Грана A из N до x из $(\Sigma \cup N)$ је постојање ако постоји $A \rightarrow \beta X \gamma$, где $\beta \gamma$ је нека анулирајућа мрежа.

3. Ако постоји и још A из N до a из Σ , тада је a елемент скупа $\text{First}(A)$.

• Ако је $x = X_1 X_2 \dots X_n$ ($X_i \in (\Sigma \cup N)$), и ако је X_i један симбол у низу који је или терминал или неанулирајући не терминал, тада је:

$$\text{First}(x) = \text{First}(X_1) \cup \text{First}(X_2) \cup \dots \cup \text{First}(X_n).$$

Пример: $S \rightarrow aSb \mid \epsilon$, најлакше граф $S \rightarrow a$ даје $\text{First}(S) = \{a\}$,
 $\text{First}(aSb) = \{a\}$, $\text{First}(Sa) = \{a, b\}$, $\text{First}(Sb) = \{a, b\}$.

Одредбене Follow(A):

Посматрамо произволну граматичку, због које имамо:

• Конструисано граф са чворовима $\sum v \{-1\} \cup N$:

1. Чвор од A у N до b у $\sum v \{-1\}$, ако постоји $X \rightarrow \alpha A \beta$ и b је $\text{First}(\beta)$

2. Чвор постоји од A у N до B у N , ако имамо паралелно $B \rightarrow \alpha A \beta$, $\beta \Rightarrow^* \epsilon$

• Када је a у $\sum v \{-1\}$ у скупу $\text{Follow}(A)$ ако постоји изи од A до a у графу.

Пример: $S \rightarrow aSb \mid \epsilon$, израда нове произведене граматике $S' \rightarrow S \mid \epsilon$,

Граф:

```

    S → b
    |
    +---+

```

Постоји граф од S до b због паралела aSb , а како постоји $S' \rightarrow S \mid \epsilon$ тако ϵ и $\rightarrow$ припада скупу, $\text{Follow}(S) = \{b, -1\}$

Одредбене скупа избора:

$C(A \rightarrow f) = \text{First}(f)$ ако f није ангуирајз

$C(A \rightarrow f) = \text{First}(f) \cup \text{Follow}(A)$ ако f јесте ангуирајз

Граматика је $LL(1)$ ако су сви ови унутар паралела дисјунтни.

Таблица прекаса

• Ово је таблица у којој сваком терминалној граматичкој укључујући ϵ одговара једна колона. Сваком не терминалној укључујући S' одговара једна линија, која ове таблице садрже паралела граматичке:

• Дакле $T[A, a]$ садржи паралелно $A \rightarrow f$ ако је a у $C(A \rightarrow f)$

```

S → E -1
E → TE'
E' → +TE' | ε
T → FT'
T' → *FT' | ε
F → (E) | a

```

Када одређујемо скуп ~~избора~~ избора за свако паралелно:

$S \rightarrow E -1$	$\text{First}\{(a)\}$
$E \rightarrow TE'$	$\text{First}\{(, a\}$
$E' \rightarrow +TE'$	$\text{First}\{+\}$
$\quad \quad \quad \epsilon$	$\text{Second}\{), -1\}$
$T \rightarrow FT'$	$\text{First}\{(, a\}$
$T' \rightarrow *FT'$	$\text{First}\{*\}$
$\quad \quad \quad \epsilon$	$\text{Second}\{+,), -1\}$
$F \rightarrow (E)$	$\text{First}\{(\}$
$\quad \quad \quad a$	$\text{First}\{a\}$

Сада можемо да нацртамо таблицу:

	a	(	)	+	*	-
S	$S \rightarrow E-$	$S \rightarrow E-$	-	-	-	-
E	$E \rightarrow TE'$	$E \rightarrow TE'$	-	-	-	-
E'	-	-	$E' \rightarrow E$	$E' \rightarrow +TE'$	-	$E' \rightarrow E$
T	$T \rightarrow FT'$	$T \rightarrow FT'$	-	-	-	-
T'	-	-	$T' \rightarrow E$	$T' \rightarrow E$	$T' \rightarrow *FT'$	$T' \rightarrow E$
F	$F \rightarrow a$	$F \rightarrow (E)$	-	-	-	-

Пример: $S' \rightarrow S-$, $S \rightarrow aSb \mid \epsilon$, таблица:

	a	b	-
S'	$S' \rightarrow S-$	-	$S' \rightarrow S-$
S	$S \rightarrow aSb$	$S \rightarrow E$	$S \rightarrow E$

• Алгоритмирајмо рад аутомата:

• Прво на стеху имамо само S' као улазни нонтерминал, у сваком кораку примењујемо:

• Ако је на врху стеха нонтерминал A , а резултни карактер таблице је a , онда пређемо дове таблице $T[A, a]$. Ако постоји улазно $T[A, a]$, онда нонтерминал A можемо претворити улазном, а ако не постоји примењујемо синхронизацију.

• Ако је на врху стеха терминал a , онда понашамо се исто као и пре. Ако је терминал a онда уклањамо a са улаза, и врху стеха, а ако је различит од a примењујемо синхронизацију.

• Празан стех значи крај рада.

Пример за $aabbb$:

Стех	Улаз	Таблица	Коментар
S'	$aabbb-$	-	(на почетку увек имамо S')
$S-$	$aabbb-$	$S' \rightarrow S-$	(на улазу "a", на стеху S' , идемо до (S', a))
$aSb-$	$aabbb-$	$S \rightarrow aSb$	(на улазу "a", на стеху $S \Rightarrow (S, a)$)
$Sb-$	$abbb-$	синхронизација	(и стех и улаз поново исти)
$aSbb-$	$abbb-$	$S \rightarrow aSb$	(симолом "a" на ња примењујемо)
$Sbb-$	$bbb-$	синхронизација	
$Ebb-$	$bbb-$	$S \rightarrow E$	(улаз "b", стех S , (S, b))
$bb-$	$bb-$	синхронизација	
$b-$	$b-$	синхронизација	
$-$	$-$	синхронизација	
ϵ	ϵ	синхронизација	

SLR(1) Граматичка таблице (3)

* LR(1) Граматика - ако левом парсирања можемо једнозначно одредити да ли је потребно извршити пребацување или редукцију, као и грамино по ком се лези, на основу једног прекудног симбола.

* SLR(1) Граматика (simple LR(1)) - подкласа LR(1) Граматика.

* Ајтем - грамино облика $A \rightarrow \alpha \cdot \beta$, где је $A \rightarrow \alpha \beta$ грамино Граматике G. Симбола је ознака процеса.

Ајтеми додељују додељу сав симбола у представљању додељу симбола неких грамина:

- Аје шатке је оно на лезу симбола
- део иза је оно што треба да пребацујемо на симбола да би могли извршити редукцију по том грамину.

Дефиницијом коначни ајтеми над $(\Sigma \cup N)^*$ за симбола процеса у представљању додељу симбола грамина левом парсирања. Овај ајтеми ће нам помоћи да формирамо и таблицу пренака.

Коначни ајтеми SLR(1):

* Симбола овог ајтемиа дје скупови ајтемиа. $S' \rightarrow S \cdot$ почетни ајтеми.

* Принципи затворења - ако за неки B из N ајтеми $A \rightarrow \alpha \cdot B \beta$ имаме неки симбола S, онда за свако грамино $B \rightarrow \beta$ граматике, ајтеми $B \rightarrow \beta$ шатке имаме S.

* Почетни симбола - симбола које шатке имаме примени принципа затворења на почетни ајтеми.

* Принципи прева - ако за неки x из $(\Sigma \cup N \cup \{\rightarrow\})$ имаме ајтеми $A \rightarrow \alpha \cdot x \beta$ имаме неки симбола S, онда ајтеми $A \rightarrow \alpha x \beta$ имаме симбола у које се преват из S по симбола x.

* Завршно симбола - симбола које садржи ајтеми $S' \rightarrow S \cdot$.

* Како би овалов ајтеми шатке за граматичку $S \rightarrow aSb \mid \epsilon$.

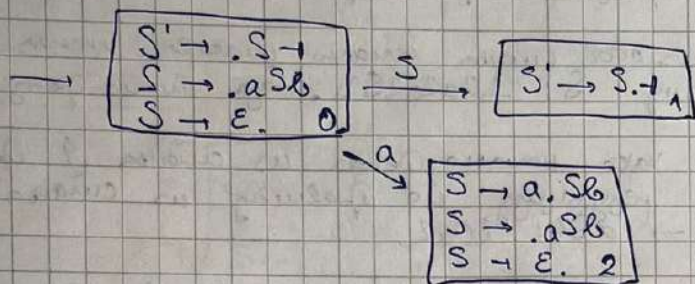
• Исходно редом овог граматичку, прво дефиницијом шатке $S' \rightarrow S \cdot$, $S \rightarrow aSb \mid \epsilon$.

• Знамо да ћемо у почетном симбола ајтеми шатке $S' \rightarrow S \cdot$, како је S унутар овог симбола шатке затворења имаме и неки додељу у ово симбола, дакле $S \rightarrow aSb$ и $S \rightarrow \epsilon$:

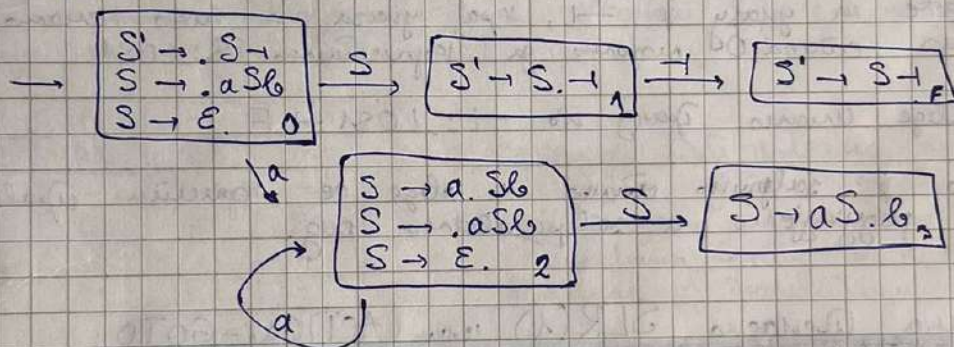
→

$S' \rightarrow \cdot S$
$S \rightarrow \cdot aSb$
$S \rightarrow \cdot \epsilon$

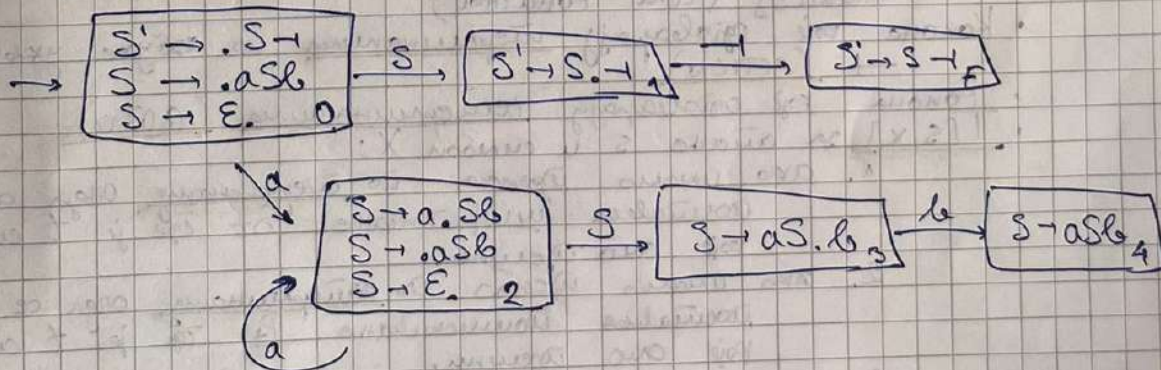
- Посматрамо преназ, из стања 0 имамо преназ S и a , преназ S добијемо неке ново стање које садржи $S' \rightarrow S \cdot 1$ а како бисмо. Нема никаквог нејединственог немамо разлику да радимо затворење за a а друге стање имамо преназ у ново стање које има $S \rightarrow a \cdot Sb$ а како имамо S нејединствено бирамо додати и два стања S нејединствено:



- Приметимо да како дођемо до стања 2 из стања 1 на главу можда преназ S и a , преназ a ћемо дефинисати као преназ a S као преназ у ново стање, што се може стања 1 можемо директно да преназ у ново стање по $\rightarrow$ у коначно стање које садржи $S' \rightarrow S \cdot 1$:



- Закључак, остаје нам још из стања 3 да наставимо даље а то радимо на сличан начин као да сада, имамо преназ a b у ново стање a $S \rightarrow a Sb$. Формално:



$\Rightarrow$ Стање F је завршно стање!

• Нап aabbb:

- Прво имаме: IO (почетак уноса)
- Због тога, одавде где можемо да идемо ако знамо да нам је S $aabbb$: $IOa2$.
- Потврђујемо исти процес, видимо да нам је S $aabbb$: $IOa2a2$
- S је b а из овог стања немамо никакве иницијалне трансформације, значи користићемо трансформацију S : $IOa2a2S3$. Сада имаме трансформацију b : $IOa2a2S3b4$.
- S је b ; а како немамо трансформацију из стања 4 преко b морали смо да извршимо редукцију по правилима из стања 4:
 $S \rightarrow aSb$: $IOa2$.
- Овде из стања 2 немамо трансформацију по b па користићемо S : $IOa2S3$.
- Коначно имаме трансформацију по b : $IOa2S3b4$
- Вршимо редукцију по $S \rightarrow aSb$: IO
- S је на улазу је -1 , крај уноса а како немамо никакве трансформације из стања 0, морали смо да користићемо S : $IOS1$.
- Одавде имаме трансформацију по -1 : $IOS1 \rightarrow F$
- Ово је завршно стање и овде се користи правило $S' \rightarrow S$ редукцијом и завршавамо рад.

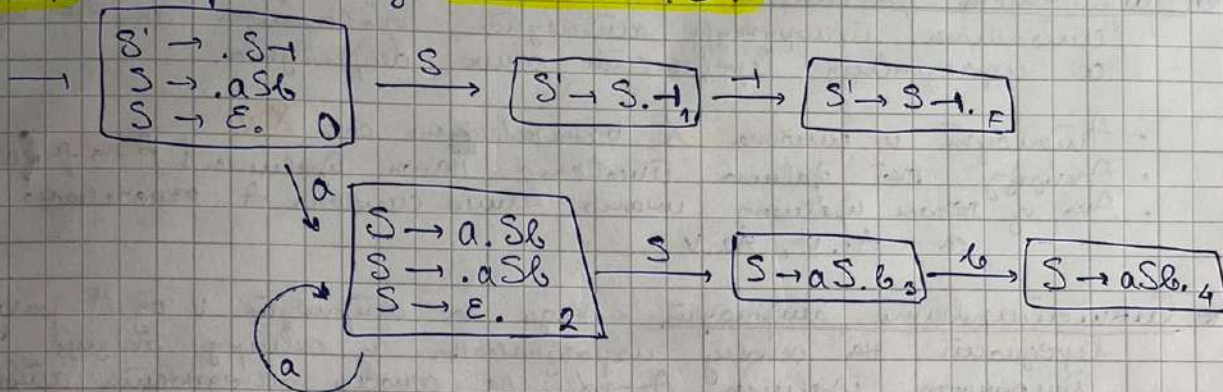
Таблица трансформације SLR(1) или ACTION-GOTO:

• Састављена је из:

- Врећа која одговарају стањима контактним аутоматима за SLR(1) анализу (сви контакти)
- Колоне које одговарају интериментима из азбуке укључујући и -1 (ACTION).
- Колоне које одговарају неинтериментима (GOTO).
- $T[s, X]$ за стање s и симбол X :
 1. ако имаме трансформацију по интерименту онда се у $T[s, X]$ поставља интеримент R_t где је t стање у које смо ишли.
 2. ако имаме трансформацију по неинтерименту онда се у $T[s, X]$ поставља интеримент R_t где је t стање у које смо ишли.
 3. ако у стању s постоји акција $A \rightarrow \alpha$, а симбол α је ϵ привага $\Sigma \cup \{-1\}$ привага окупи Follow(A), онда у $T[s, X]$ уписујемо акцију редукције по правилима $A \rightarrow \alpha$, R_i где је i редни број правила које редукцијом у приваги G .

⊛ Конфликтни настатјат ако кага на исто место неможе да
 извршимо shift и reduce (shift-reduce конфликт) или кага
 неможе да извршимо два reduce (reduce-reduce
 конфликт), така граматика неје SLR(1).

Пример за граматика $S \rightarrow aSb \mid \epsilon$:



0: $S' \rightarrow S -1$
 1: $S \rightarrow aSb$
 2: $S \rightarrow \epsilon$ Follow $\{b, -1\}$

Како се једноко до S резултије, негасно Follow(S) = $\{b, -1\}$, домаифасно
 резултијуј само тих конона:

	a	b	-1	S
0	S2	R2	R2	P1
1	-	-	SF	-
2	S2	R2	R2	P3
3	-	S4	-	-
4	-	R1	R1	-

Резултије употребасно негасјути
 резултије гласне групас ситак
 које негасно иа у конона симбона,
 употребасно резултије да R_i за i -то
 употребасно у граматичу.

Пример израза са $+$, $*$, $($, $)$ / pdf. 33-39

Семантичка анализа

Атрибутивна граматика

• Атрибутивна граматика - контекстно слободна граматика у којој се симболима придружују атрибути, граматичке акције којима се израчунавају вредности ових атрибута.

- Атрибут v симбола X означавамо са: $X.v$
- Акцију неке граматичке писаче након граматичке: $A \rightarrow \alpha$ {акција}
- Ако у неким граматичким имамо више симбола A означавамо га са: $A_1.v, A_2.v, \dots$

• Синтектизовани атрибути - када за атрибут v од A рачунамо вредности на основу израчунавања у оквиру акције придружене граматичкој $A \rightarrow \alpha$, на основу вредности атрибута из α .

• Граматика је S-атрибутивна ако су сви атрибути у којој синтектизовани.

Пример за граматичку израза:

$$\begin{array}{ll} E \rightarrow E + T & \{ E_1.v = E_2.v + T.v \} \\ E \rightarrow T & \{ E.v = T.v \} \\ T \rightarrow T * F & \{ T_1.v = T_2.v * F.v \} \\ T \rightarrow F & \{ T.v = F.v \} \\ F \rightarrow (E) & \{ F.v = E.v \} \\ F \rightarrow a & \{ F.v = \text{val}(a, \text{lex}) \} \end{array}$$

Пример израчунавања за $a + a * a$, где редом одговарају 2, 3 и 5:

...

Симболно алуиракитне синтиаксе

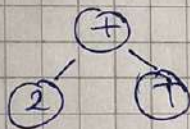
• Зодуја се као крајњи циљ синтиаксне анализе.

• Ово симболно је садржи конкретне синтиаксе језика дојди саграда сепааратора и кбухних речи, садржи само оне информације које су неопходне са симболним синтиаксе или значења одговарајуће језичке конструкције.

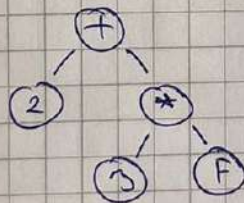
$E \rightarrow E + T$	$\{ E_1.k = \text{new_bin}(' + ', E.k, T.k) \}$
$E \rightarrow T$	$\{ E.k = T.k \}$
$T \rightarrow T * F$	$\{ T_1.k = \text{new_bin}(' * ', T.k, F.k) \}$
$T \rightarrow F$	$\{ T.k = F.k \}$
$F \rightarrow (E)$	$\{ F.k = E.k \}$
$F \rightarrow a$	$\{ F.k = \text{new_coef}(a.\text{lex}) \}$

Пример $a + a * (a + a), 2 + 3 * (5 + 6)$:

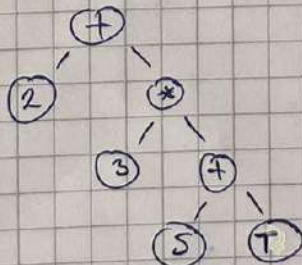
• Како је први елементи a , први догађено a као мести. Снегети симбол је $+$ на пралимо налим клор сингаре операције са $+$ и четомо. Догађено вередности израза који је у овом моменту само T као десни операнд:



• Затим домаитрамо T , како лудимо да имамо a , пралимо налим мести a снегети симбол је $*$, дакле пралимо налим клор за операндор $*$:



• Снегети елементи је саграда, нуде дилна за напалавано. Умало да је снегети елементи a пралимо налим мести, одмах након је $+$ на пралимо налим клор са $+$:



• Коначно пралимо налим мести a :

